Bisleri Bottle Detection using YOLOv8

---

1. Objective

The objective of this project is to build a robust object detection model capable of identifying Bisleri bottles from images and video feeds in real-world scenarios. The model should detect bottles under varying angles, distances, and lighting conditions, making it suitable for surveillance, quality control, or automated inventory systems.

---

2. Requirements

2.1 Hardware Requirements

- GPU-enabled system (NVIDIA T4/A100 or equivalent) for efficient training
- Minimum 8GB RAM
- Storage for datasets and model weights (~5GB)

2.2 Software Requirements

- Python 3.10–3.12
- Google Colab or local Jupyter Notebook
- Libraries: ultralytics, roboflow, supervision (for dataset management and evaluation)
- Roboflow account (optional for dataset upload/augmentation)

2.3 Dataset Requirements

- Custom-labeled dataset of Bisleri bottles
- Minimum 1000 images recommended
- Dataset structure:

```
train/
    images/
    labels/
val/
    images/
    labels/
data.yaml
augmentations.yaml
```

- Labels should be in YOLO format

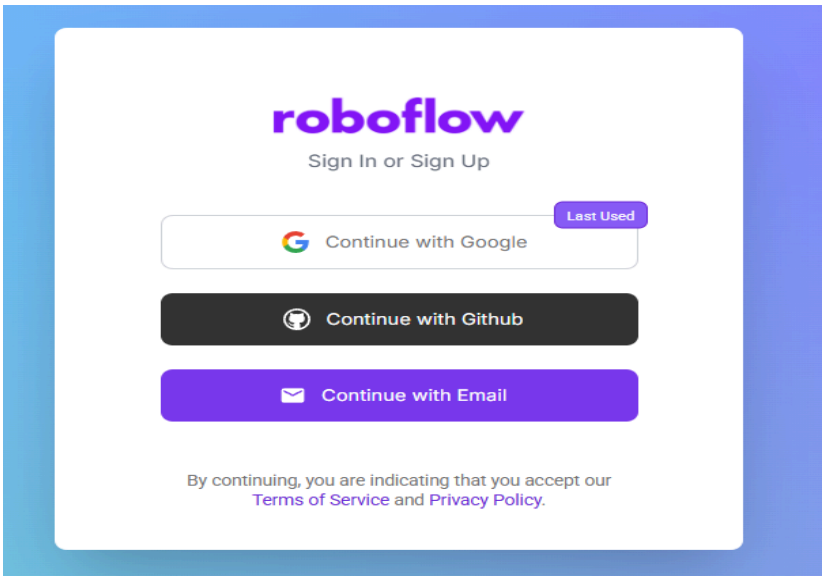
- Images should cover multiple angles, scales, and lighting conditions

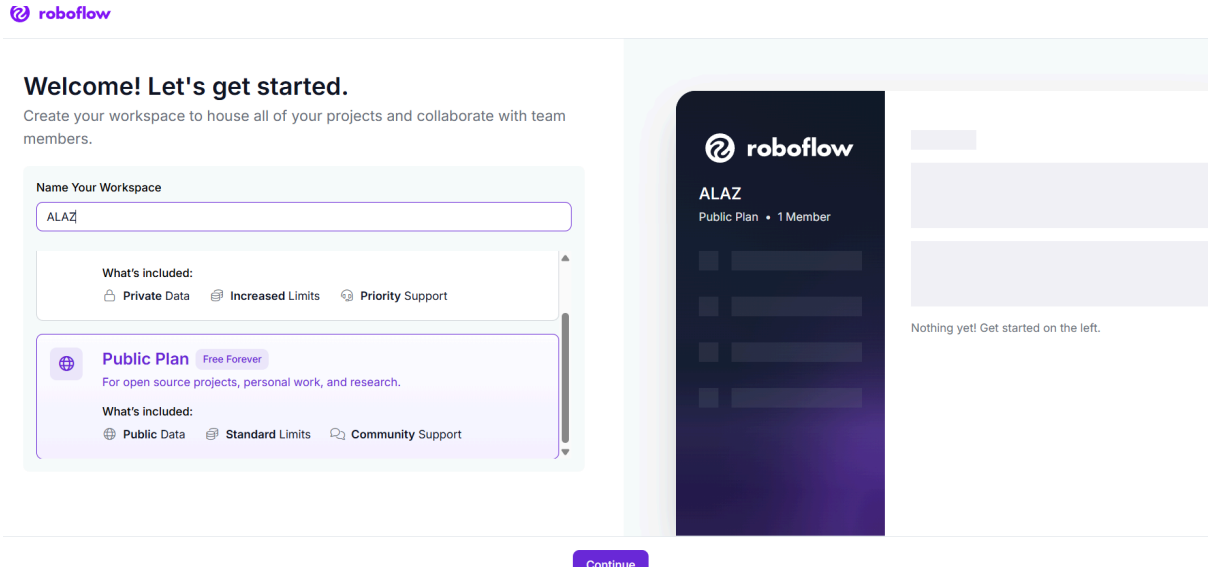
---

3. Methodology

3.1 Data Preparation

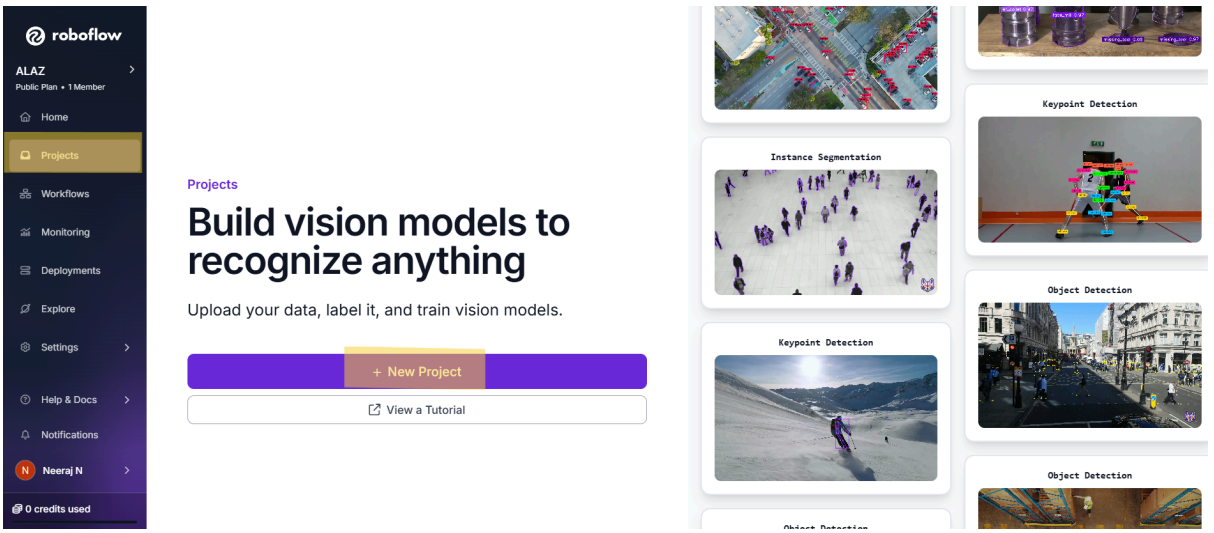
- Login in to Roboflow and signup to an account



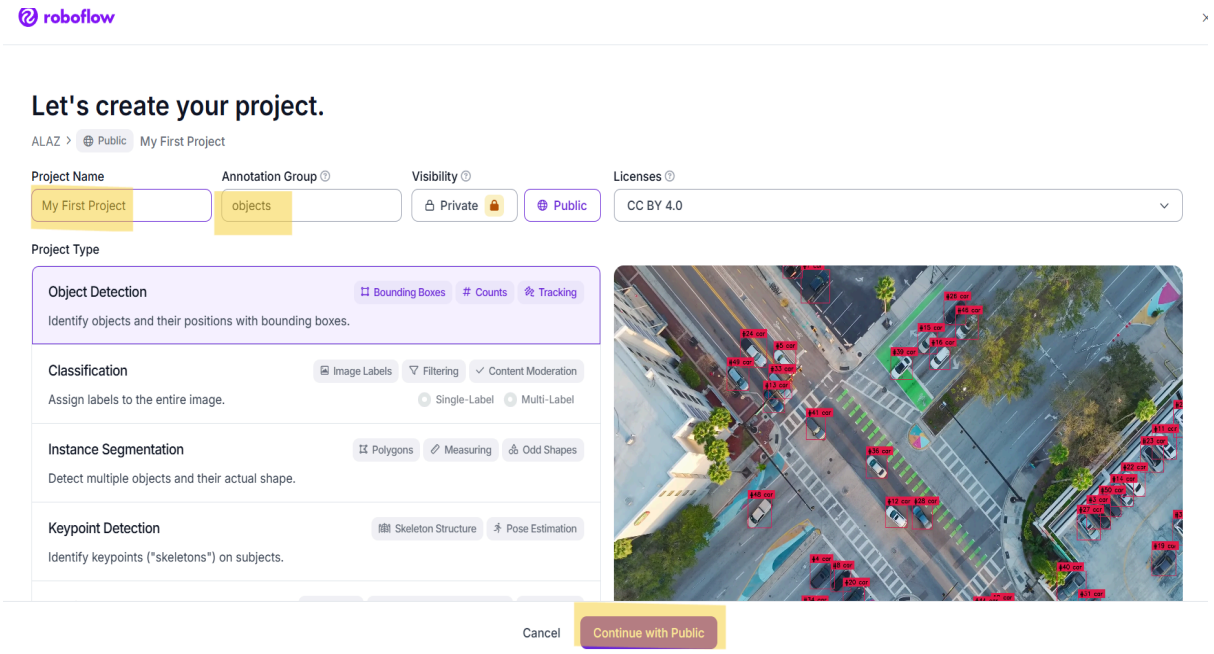


- Create a new project then add a name and choose object detection model. This are the below steps

  i>>> First click on projects and then click “+ New Project”

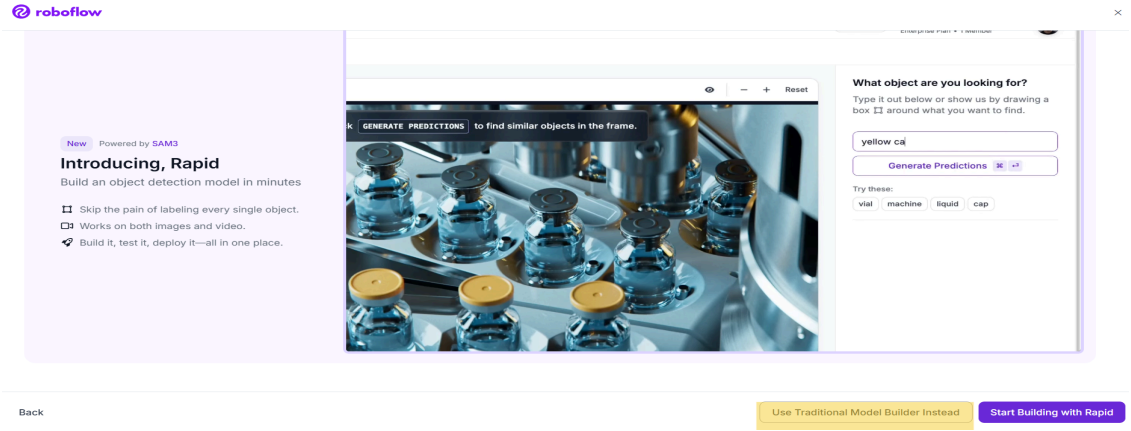


ii>>> After that write project name and annotation group name like below picture and select the project type as object detection then continue with public

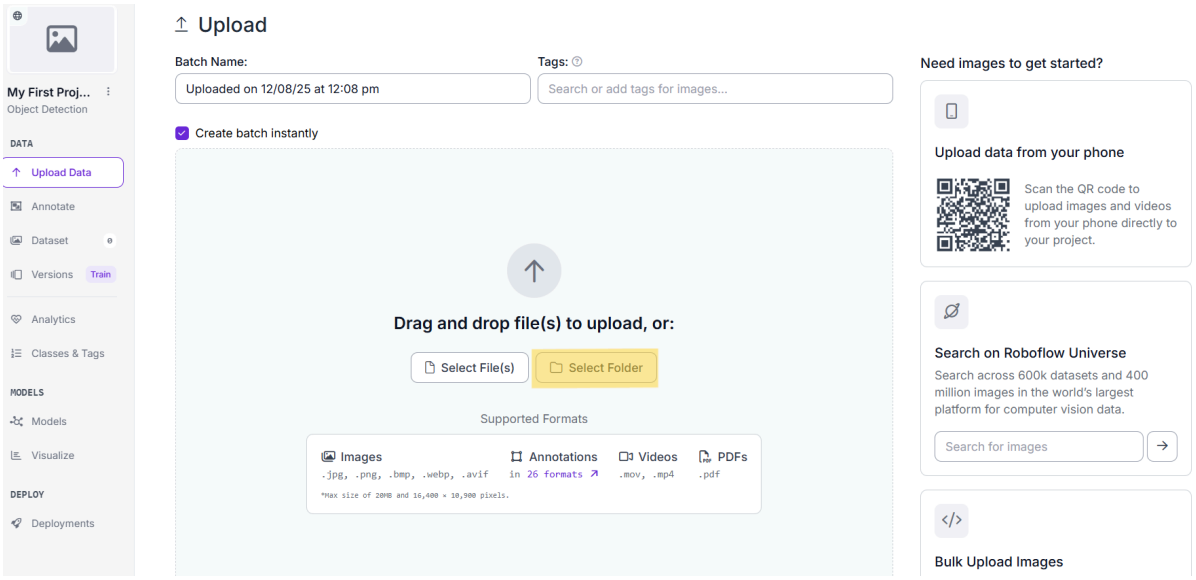


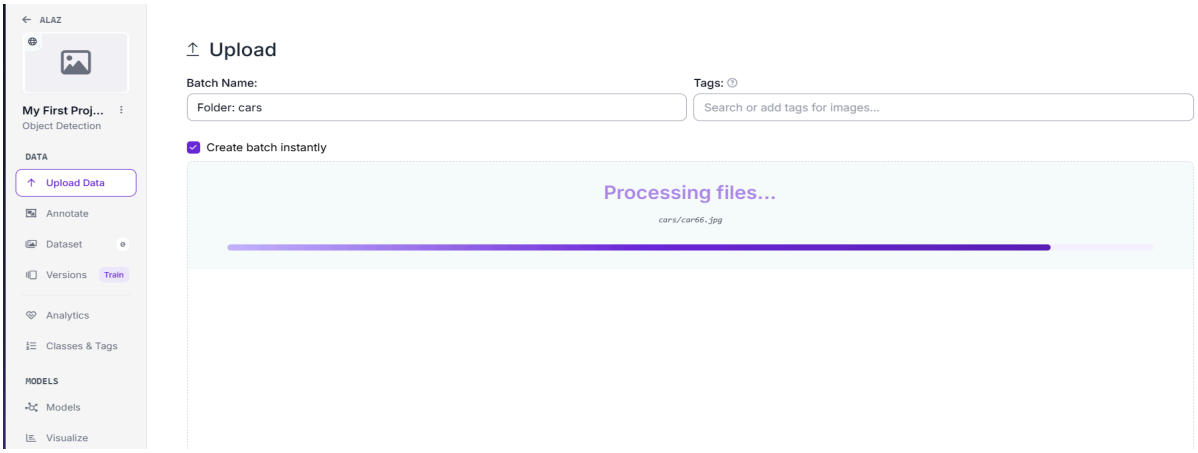
iii>>> We can “start building with rapid” is also be used as ai can detect objects with small amount of data for small objects and fast processing speed

iv>>> But for large object detection we use “ Use traditonal model buider instead “



v>>> Next after all above steps it will direct to this page, then click on **Select folder** for uploading the images



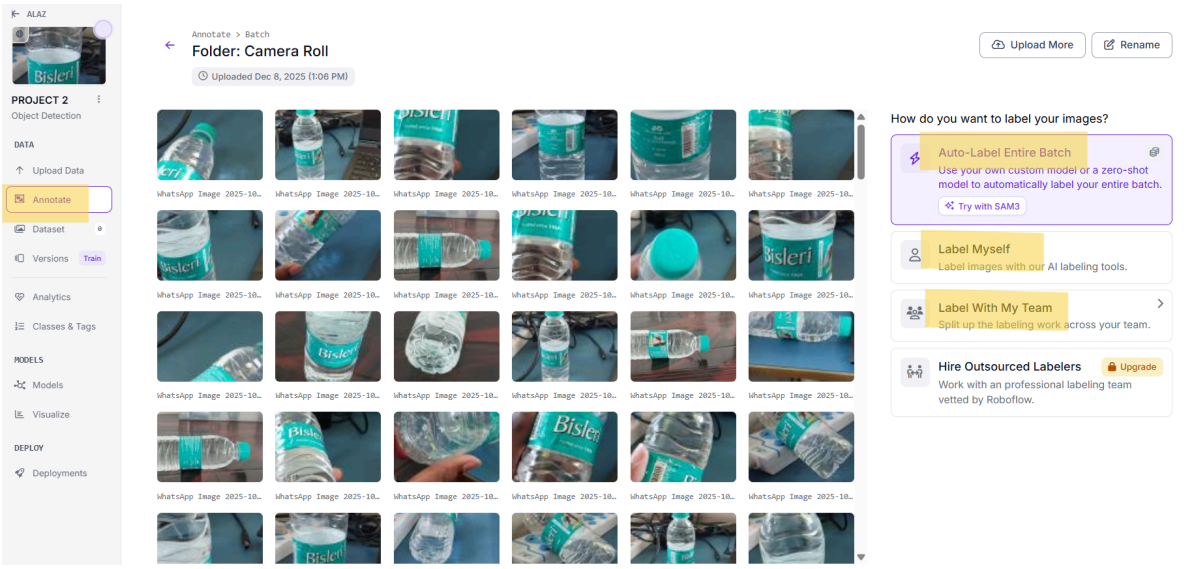


vi>>> After uploading the images then go to 2nd option Annotate where it is highlighted and we can see 3 options highlighted under it

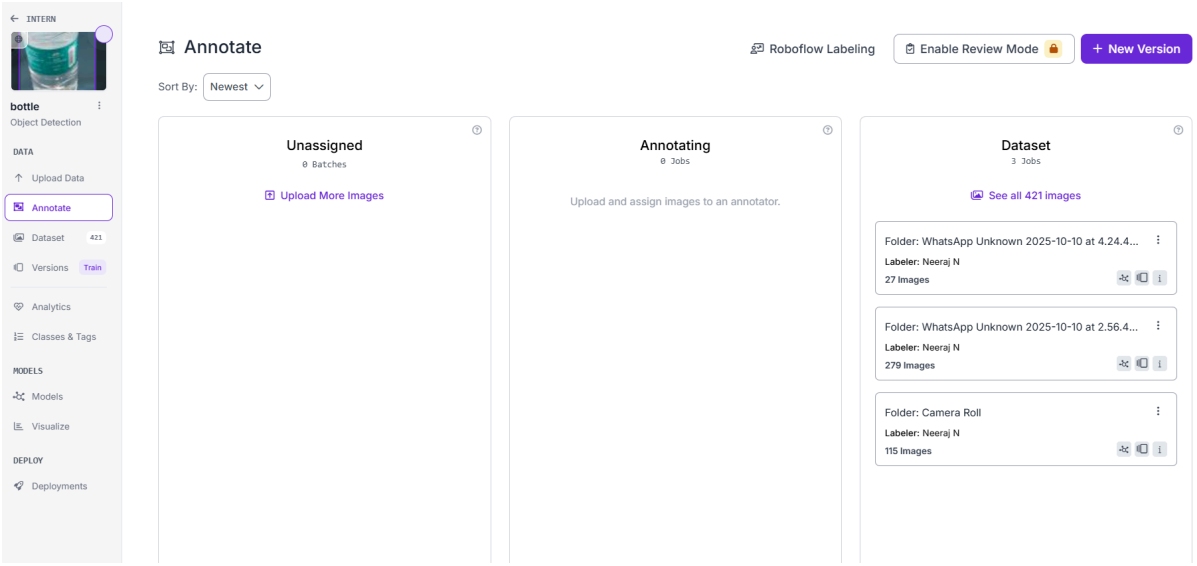
1> Auto label entire batch : it is using ai while we annotate some image with labeling it will take the pattern and annotate similar kind image with labeling it so it will help us to complete the work on time

2>Label Myself : to do labeling the image manually with some help of ai

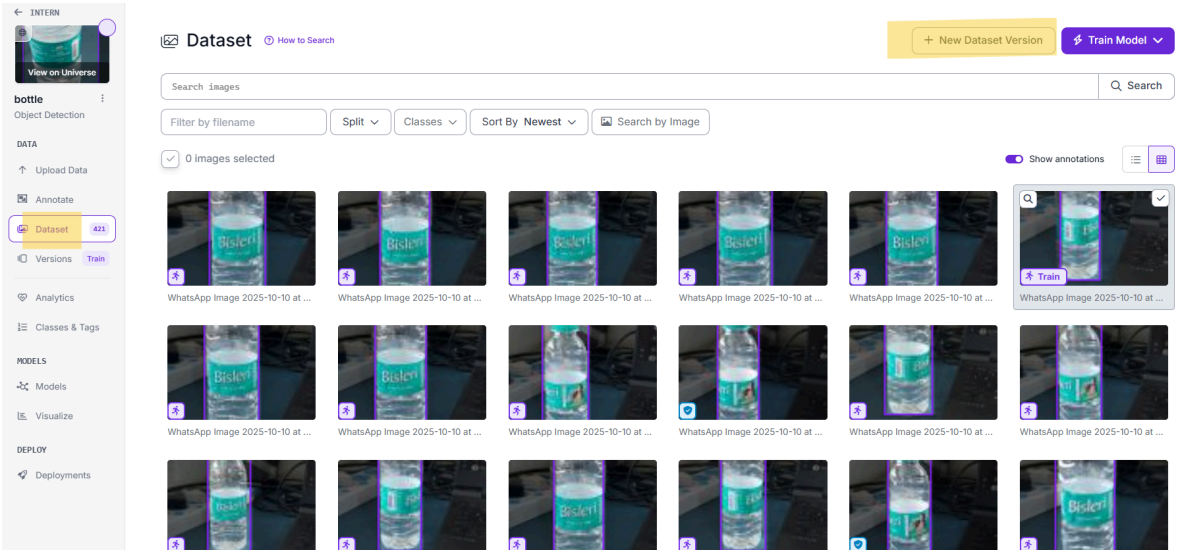
3>Label with My teams : we can add some friends email ids for manually label the images



4>After annotate or labeling images the folder will come to the dataset



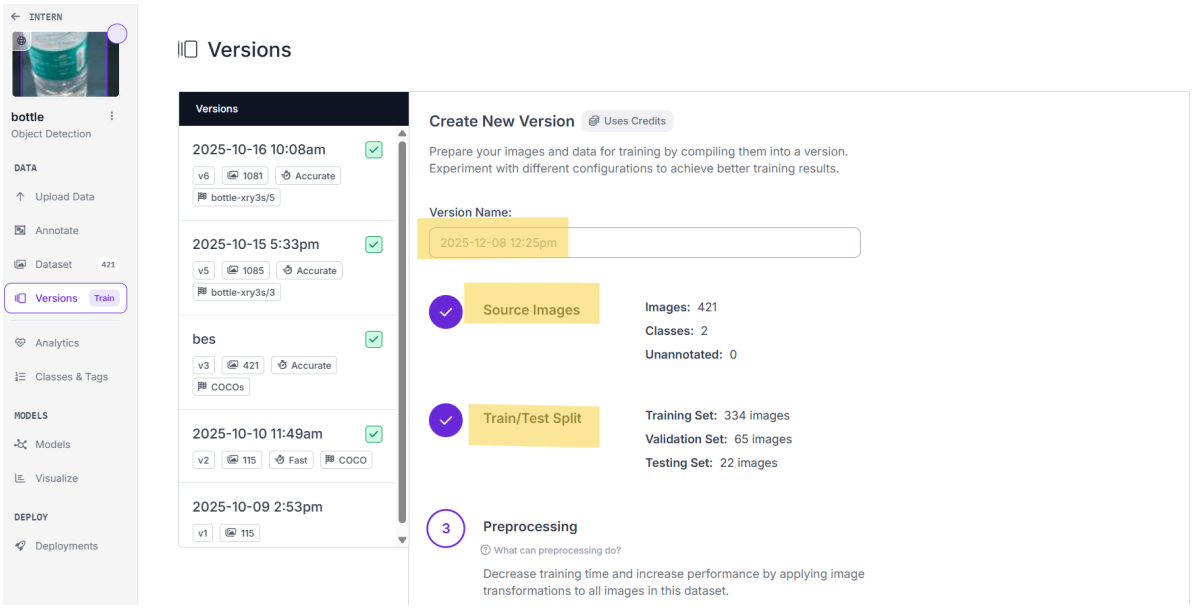
then click on " **Dataset "** so we can see another button called " + New Dataset Version" so it will directly go to the versions

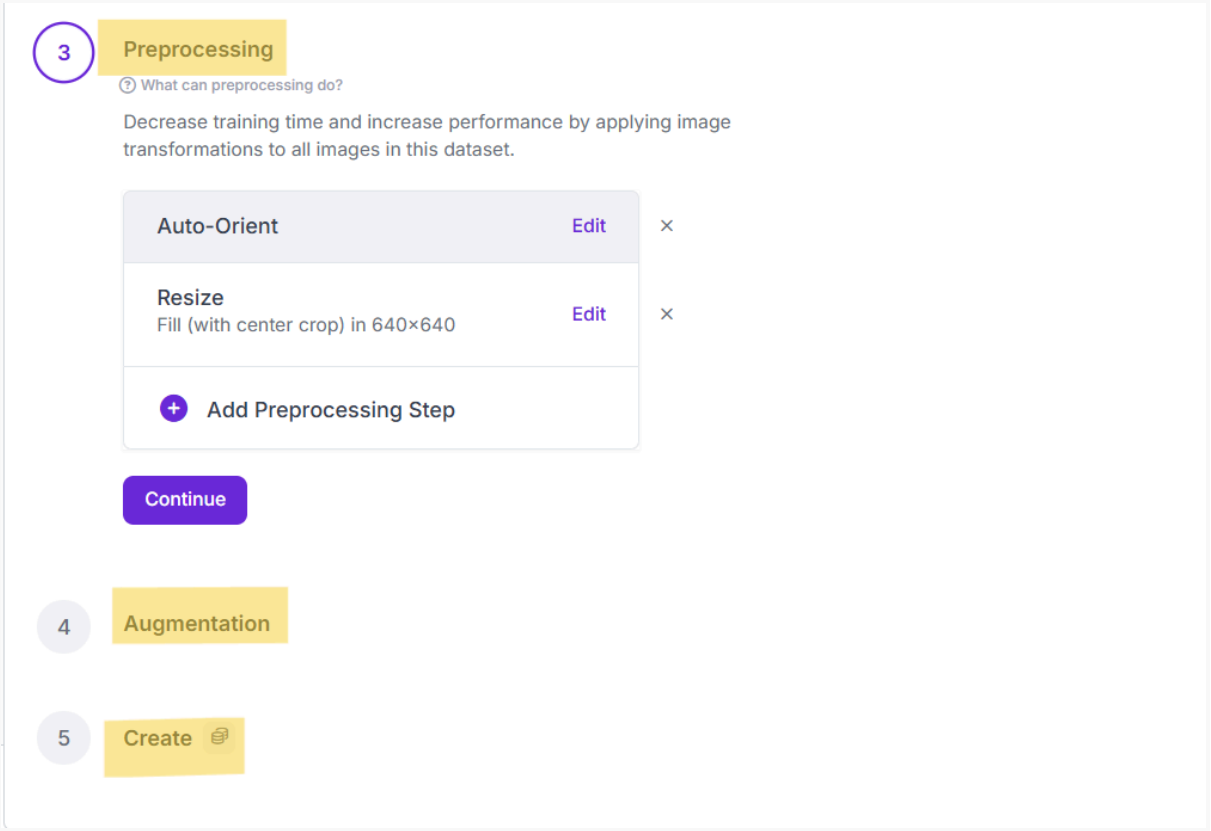


3.2 Model Selection

- YOLOv8 chosen for state-of-the-art object detection
- YOLOv8n for fast training, download YOLOv8n/s/m for higher accuracy
- Use pretrained weights to leverage transfer learning

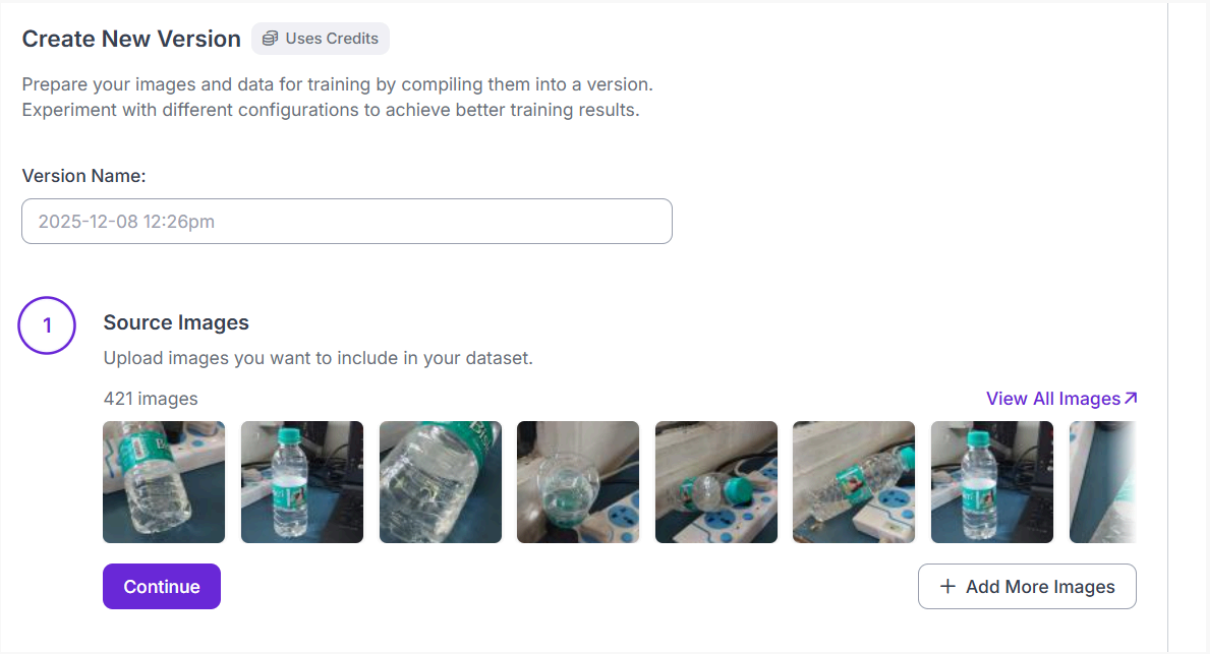
3.3 Training Configuration



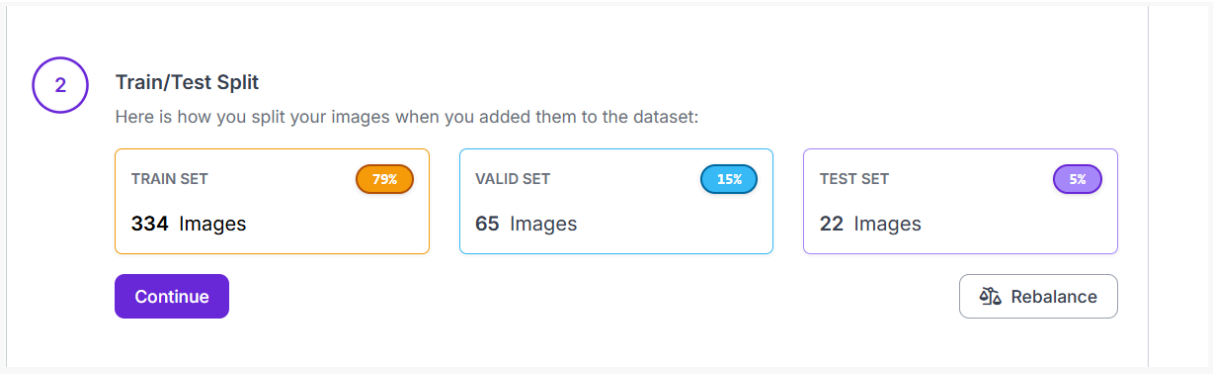


In that we can create new version there are 6 steps to proceed:

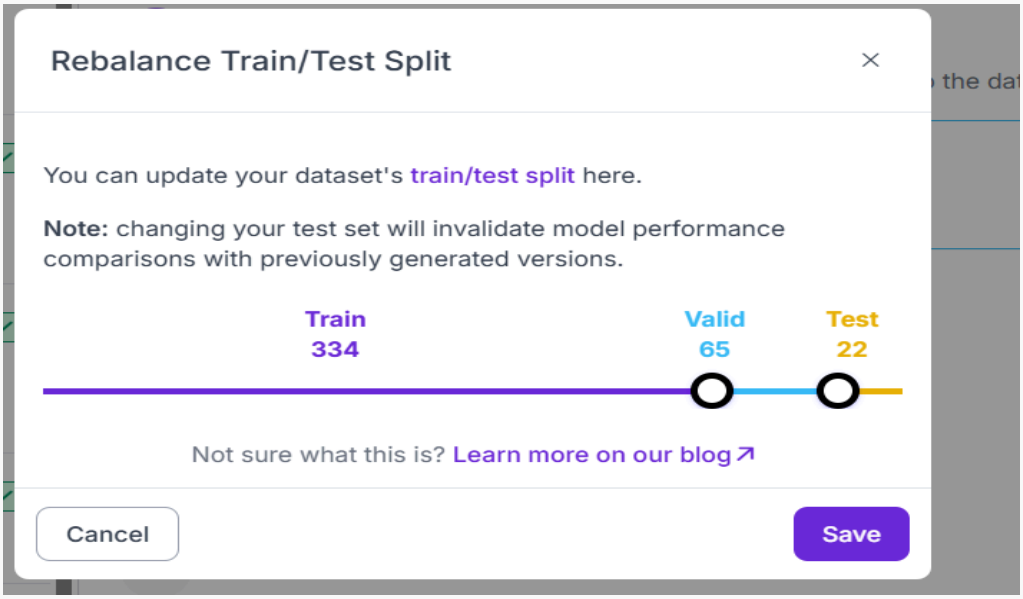
>> Step 1: Name the version
>> Step 2: Edit or check the source images of the dataset and can upload more images



>> Step 3: Edit Train/Test Split for training/validation/testing for better user divide into (**80% for training, 10% for validation, 10% for testing**)

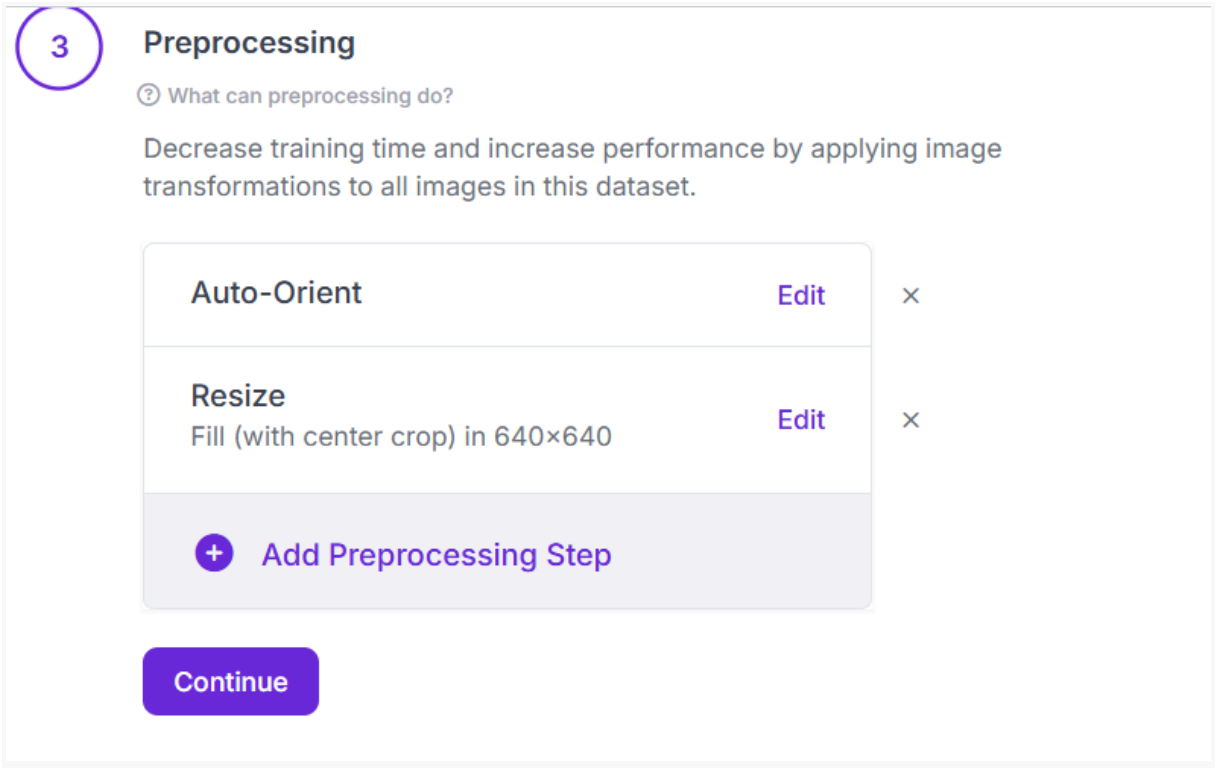


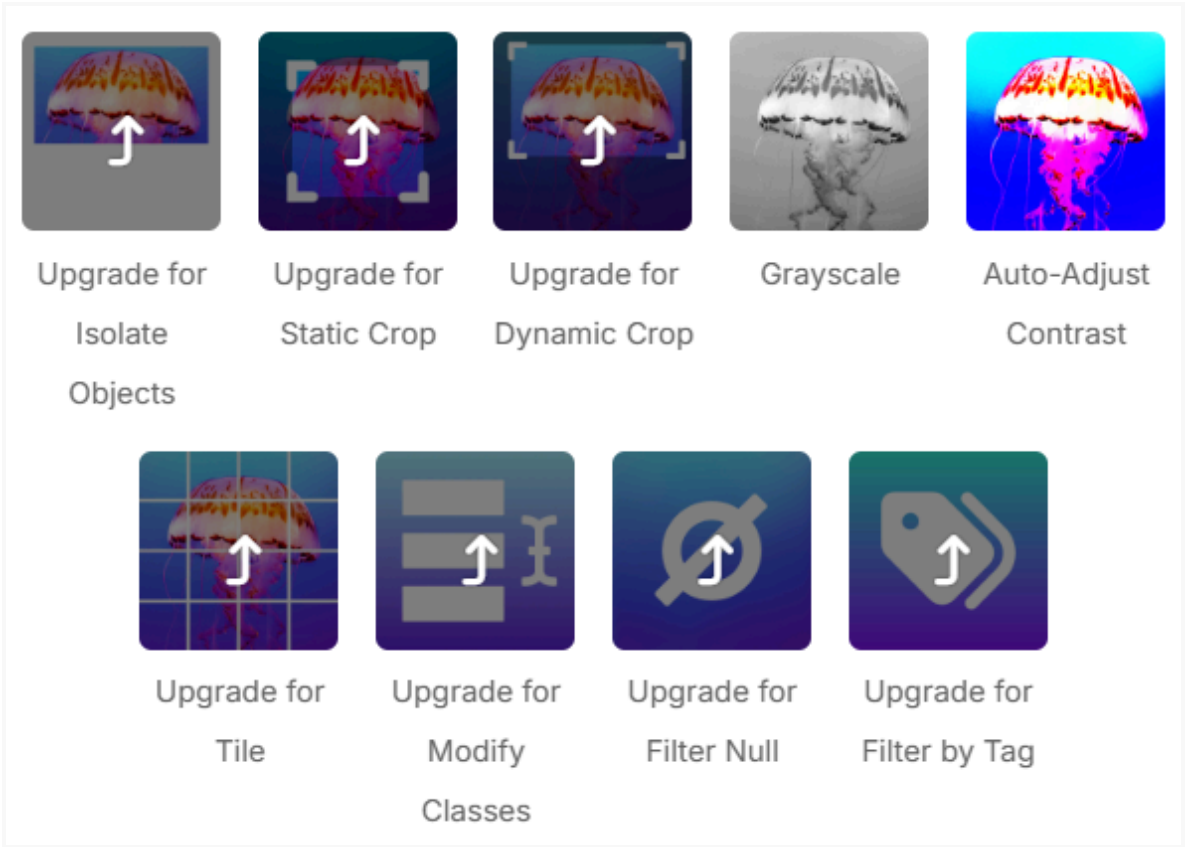
if we press rebalance it will adjust images sets



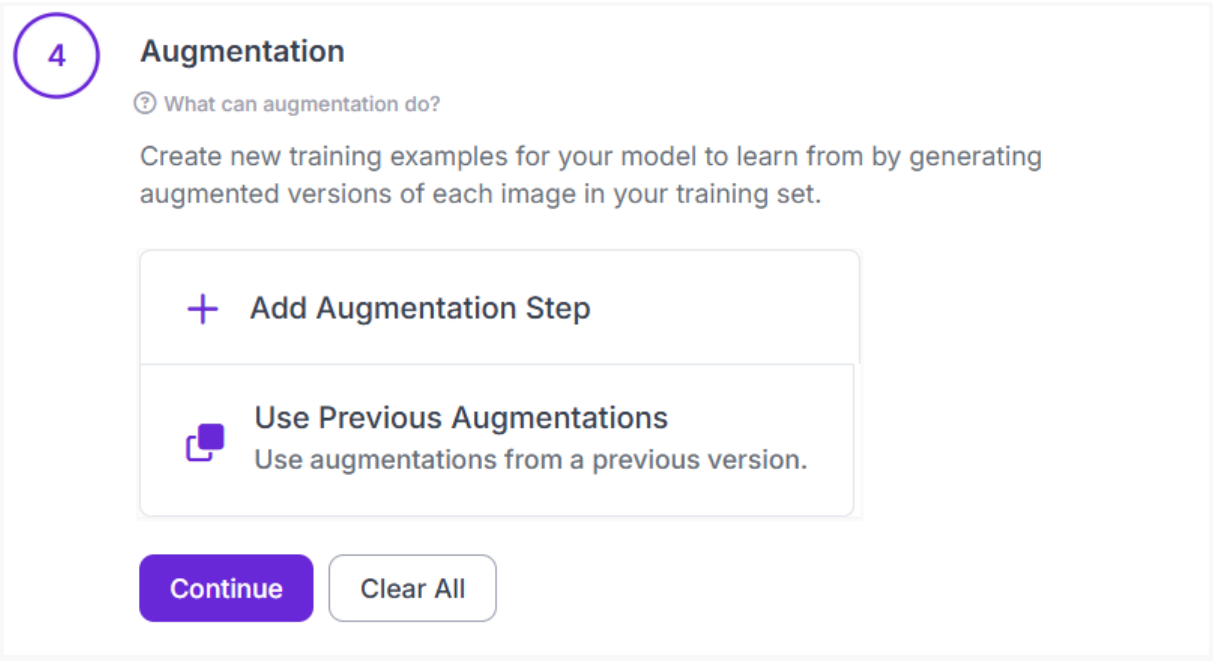
>> Step 4: In preprocessing click on **Add preprocessing step** so we can see preprocessing parameters to auto orient , resize the images and grayscale for Yolov8 to train the model and then we have to press continue to save the steps.

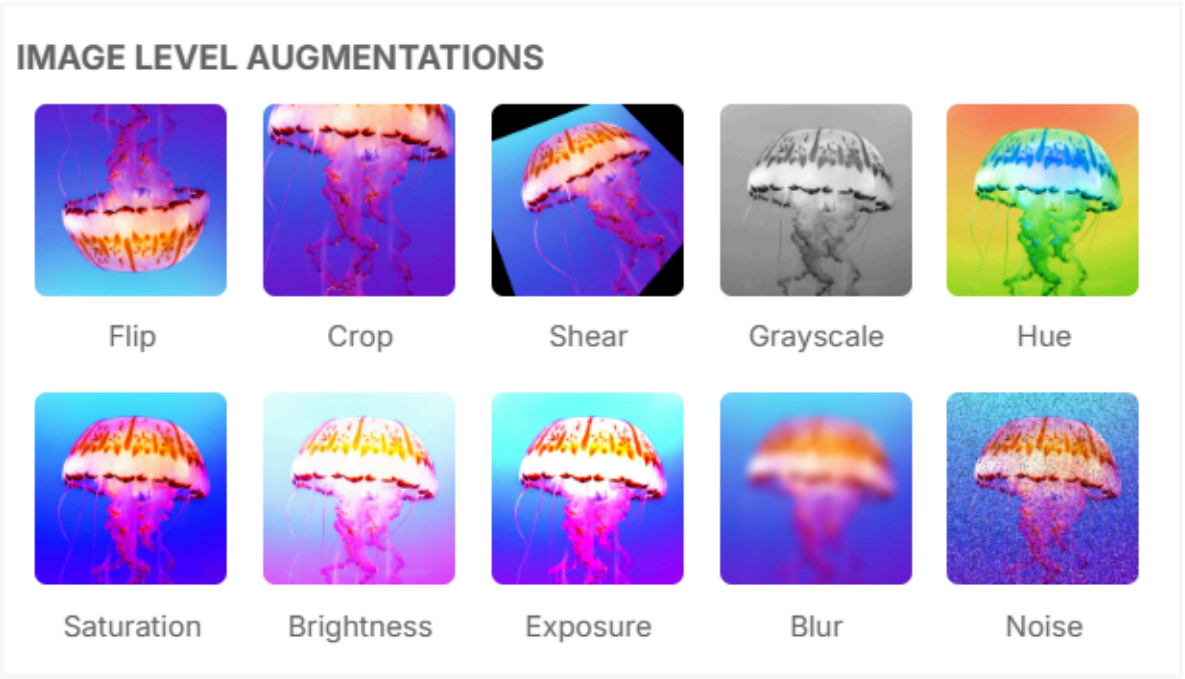
(we can add more parameters in upgrade version)





>> Step 5: In Augmentation click on **Add Augmentation step** so we can see augmentations for Yolov8 to train the model and then we have to press continue to save the steps.

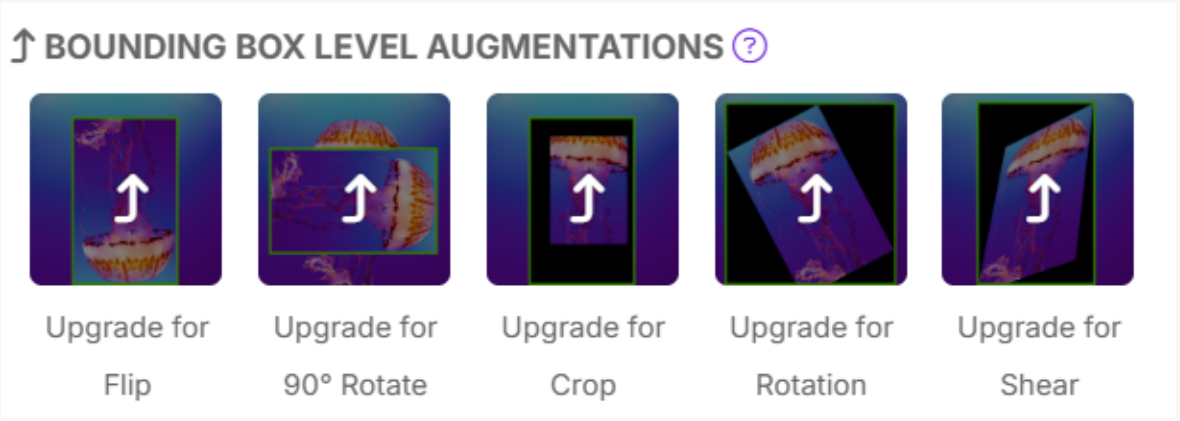




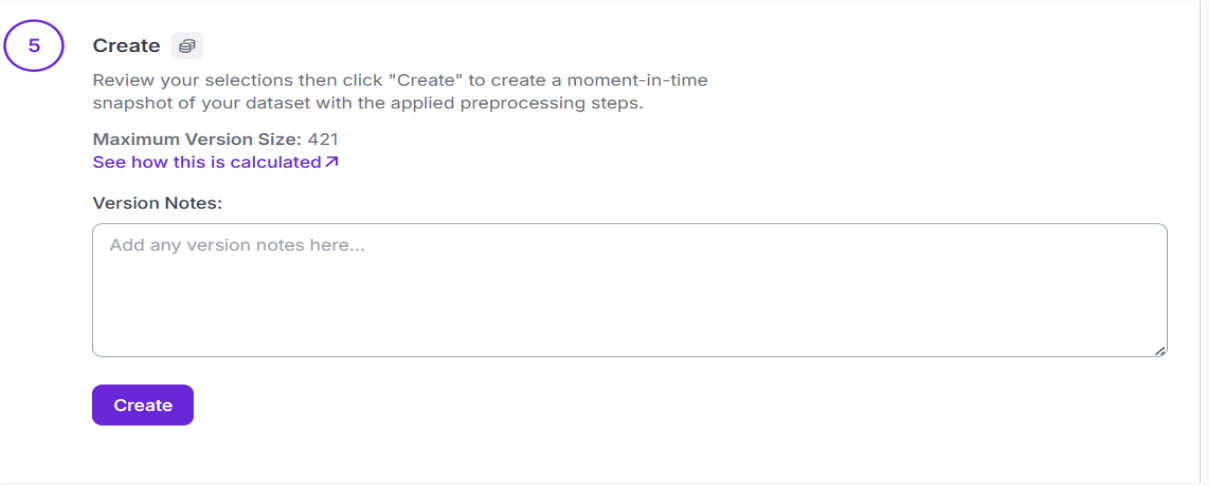
- Data augmentation:
  - Rotation: ±20–45°
  - Translate: ±10–20%
  - Scale: 0.2–0.5
  - Flip horizontal/vertical: 40–50%
  - HSV augmentation for brightness, contrast, saturation

- Mosaic & Mixup for additional robustness

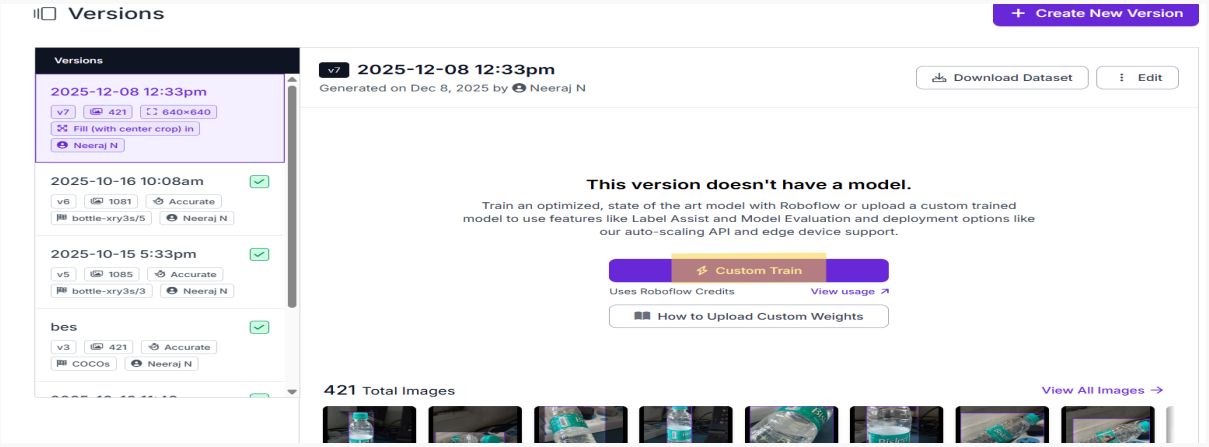
- If you want more augmentations please upgrade



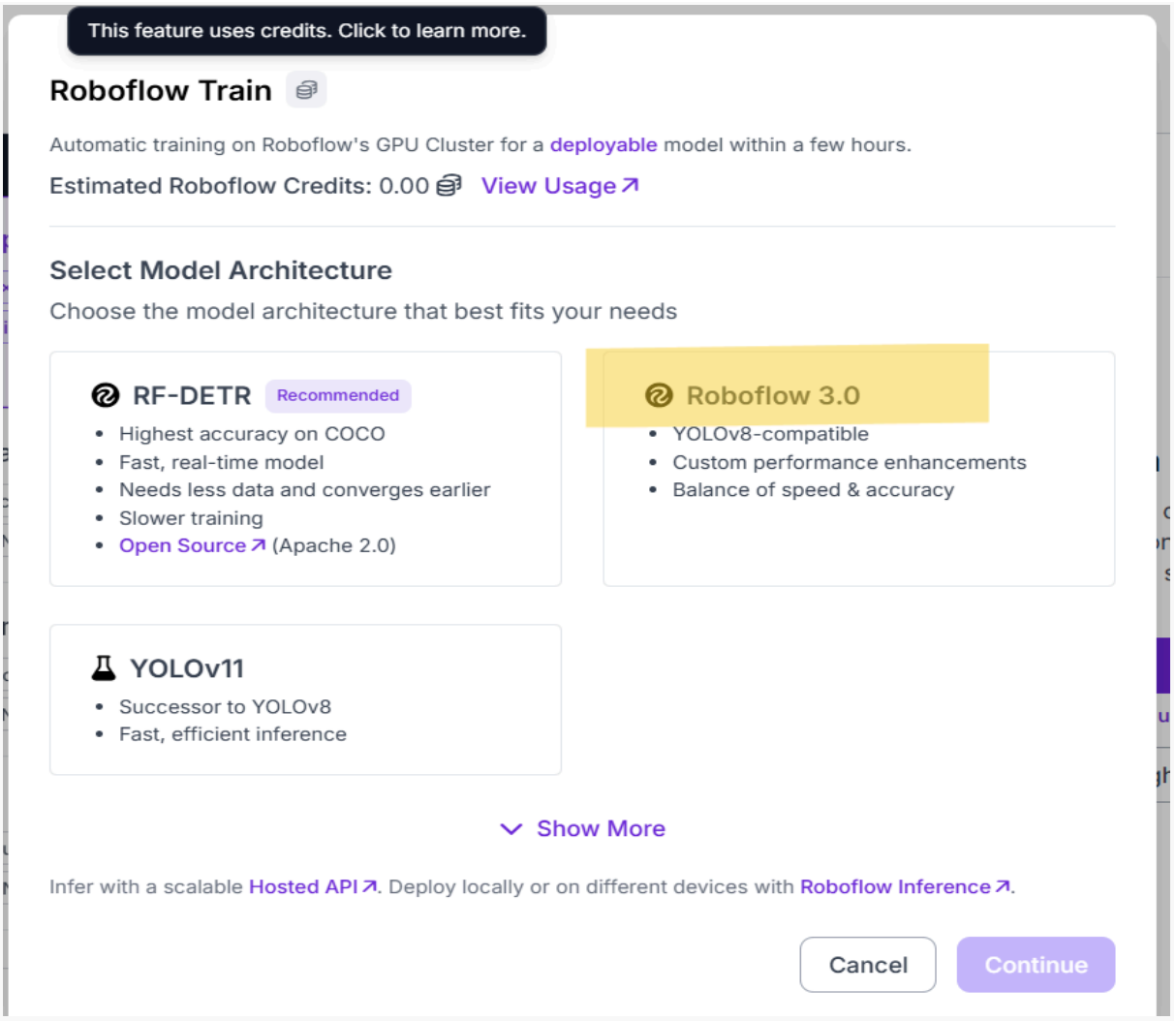
>> Step 6: click on create.



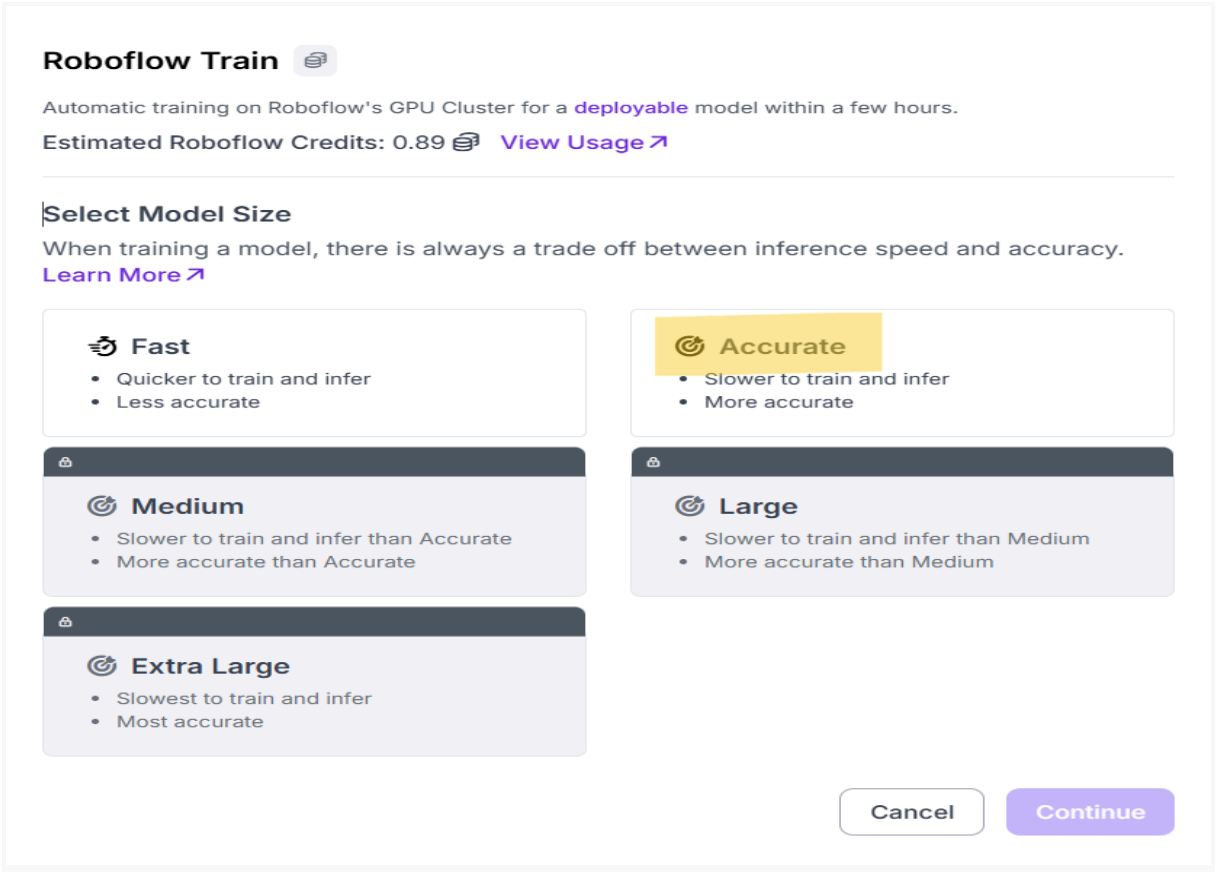
After creating the version in the middle of the page click on custom train



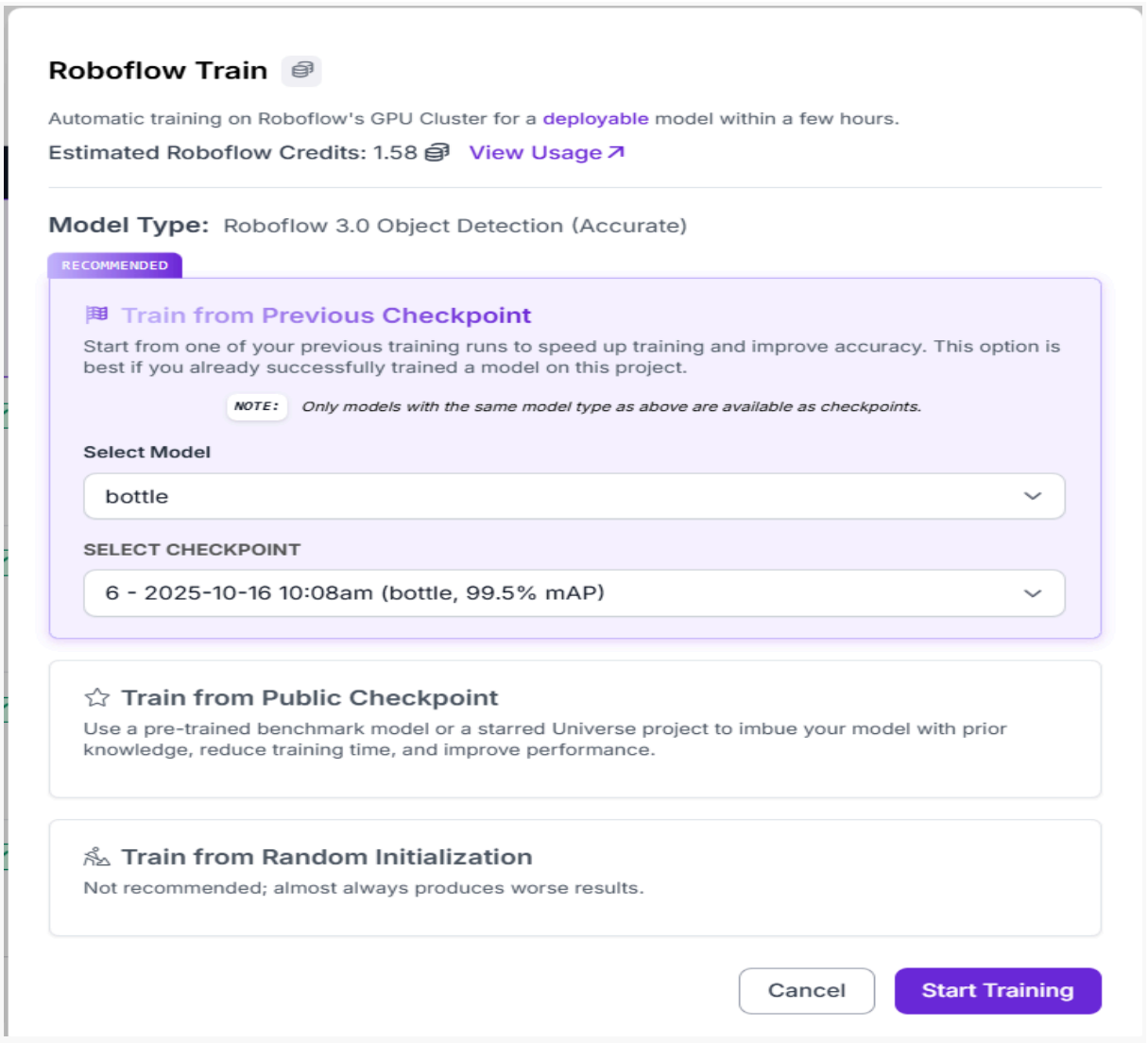
Click on Roboflow 3.0 and press continue



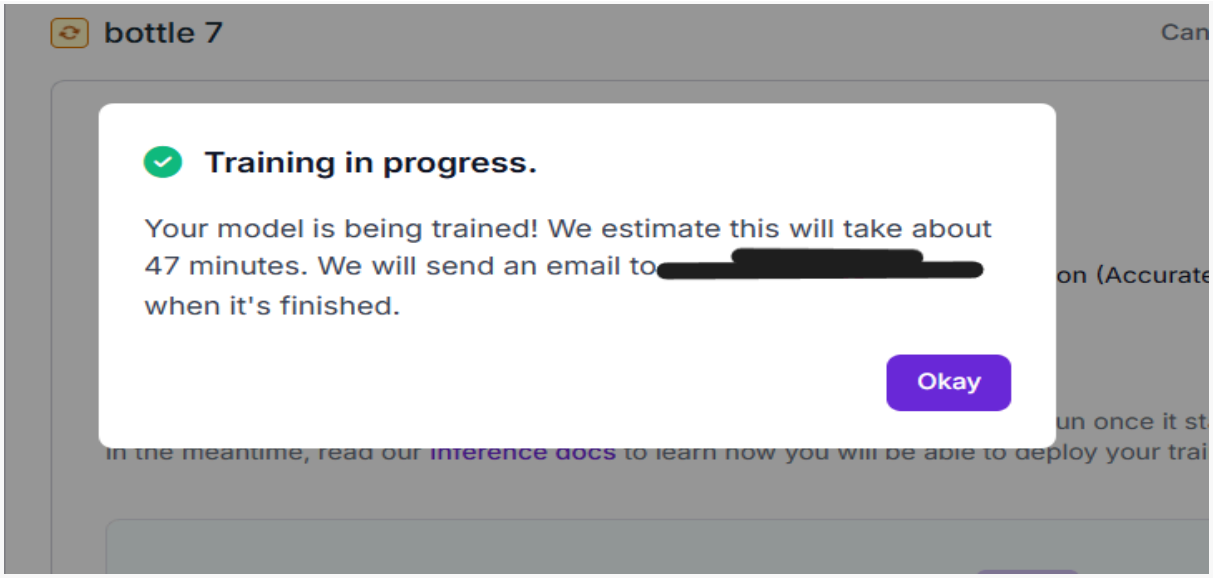
Next it will take select model size there click on accurate and continue



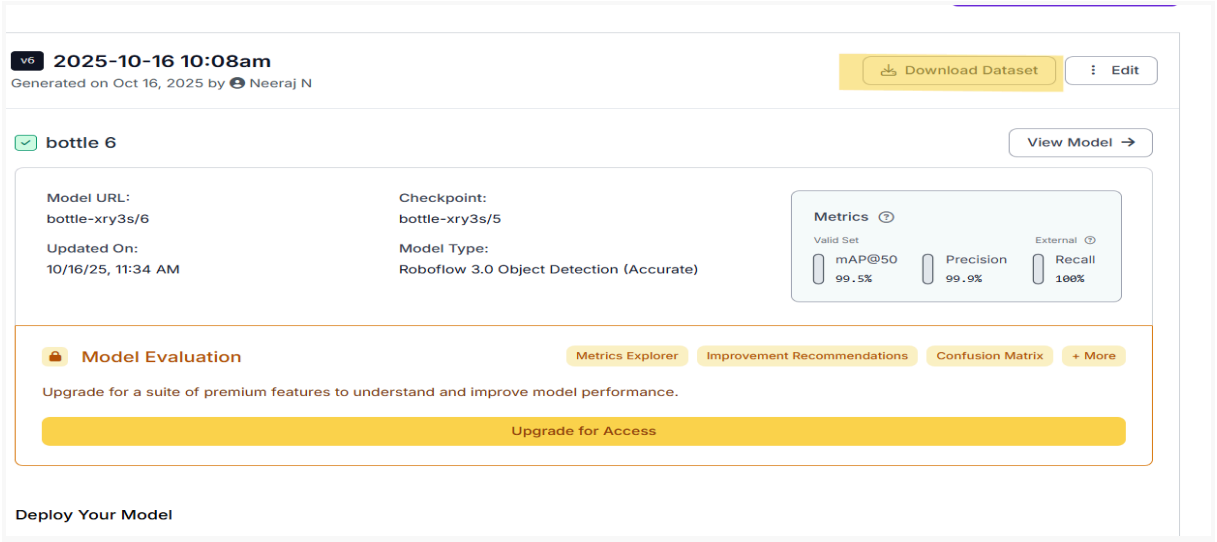
This is the next step where we can recommend any best model in our project best model in **SELECT MODEL** and with its version in **SELECT CHECKPOINT** where the roboflow train can take its best model reference as we say like this the model should be trained and click on start training

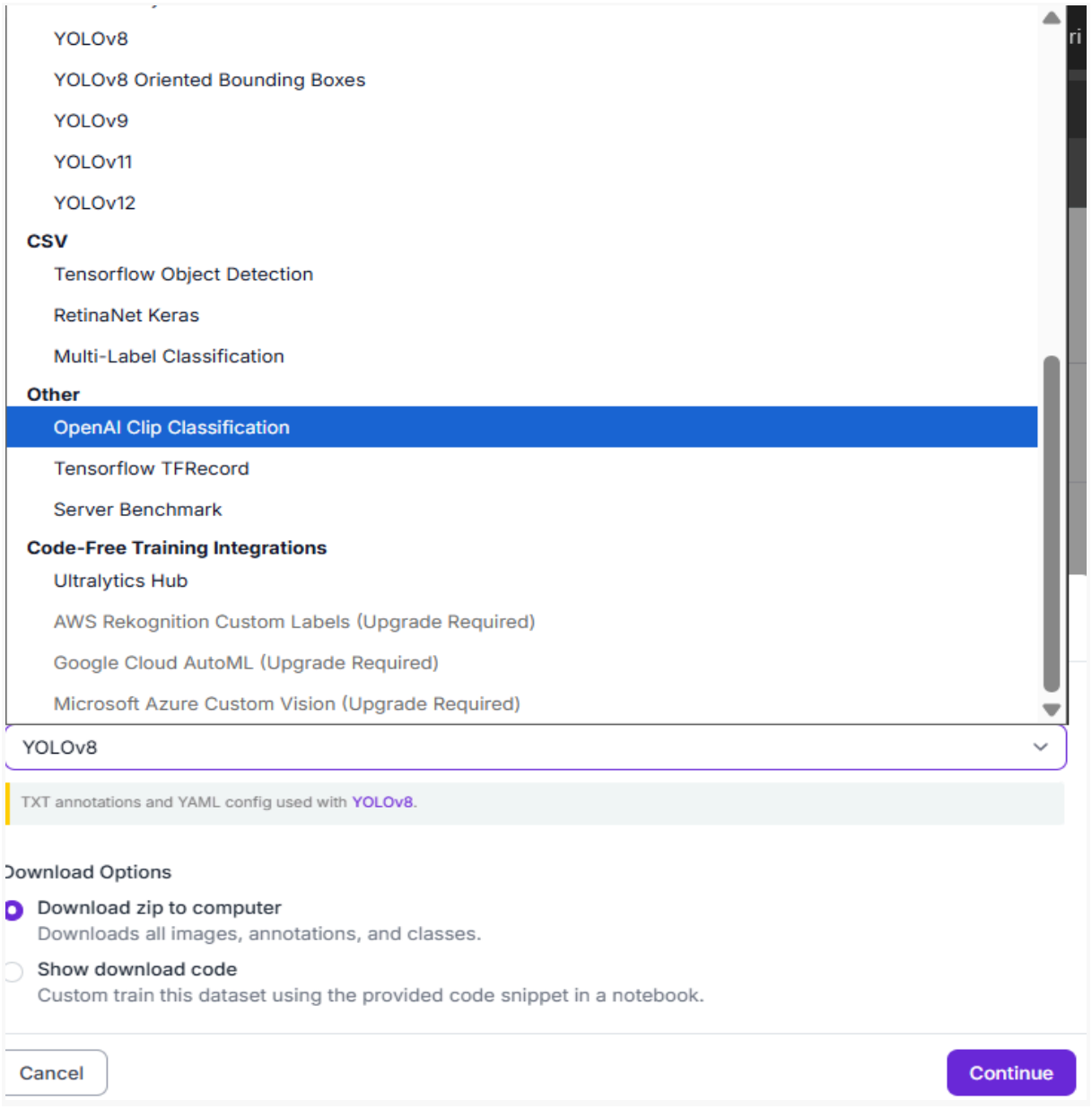


At last this message will pop out to take some for model training and evaluation

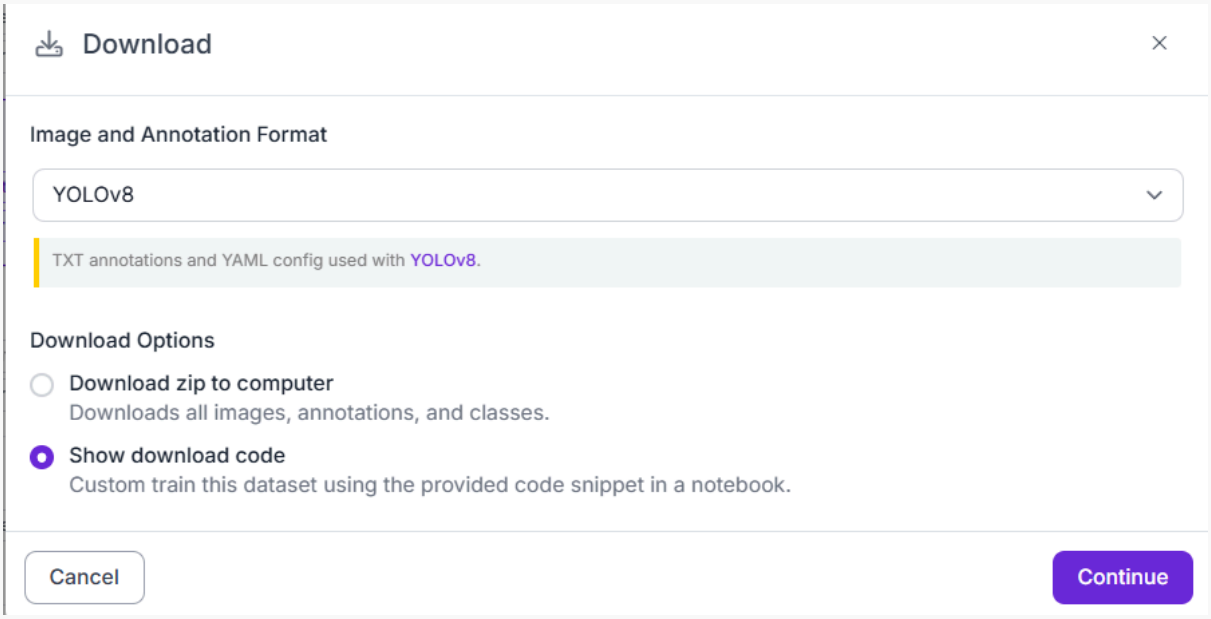


After completing the training click on **download dataset** and choose only Yolov8 format and continue

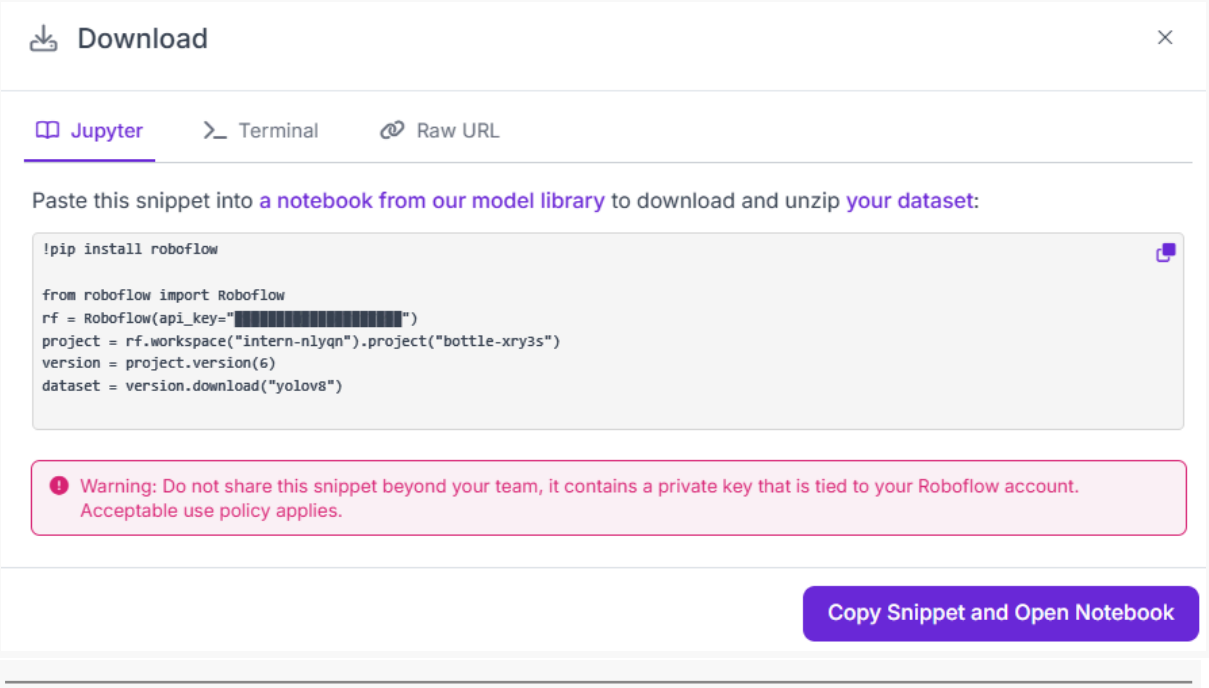




click on the show download code -> continue.



Then choose Jupyter and copy the code snippet



4. Implementation Steps

Now we can run the model in the training folder to check the accuracy and loss of the model.

So we are using google colab notebook to train the model.
This are the commands to run the model :

```
!pip install ultralytics
!pip install roboflow
```

```
import ultralytics
from roboflow import Roboflow
from ultralytics import YOLO
from IPython.display import Image
```

paste your jupyter code snippet here like this

```
rf = Roboflow(api_key="P5NeSKFmyHHNd7wrVe9p")
project = rf.workspace("intern-nlyqn").project("bottle-xry3s")
version = project.version(2)
dataset = version.download("yolov8")
```

```
!yolo detect train data=/content/bottle/data.yaml model=yolov8n.pt epochs=50
imgsz=640

Image(filename=f'/content/runs/detect/train/results.png', width=600)

!yolo task=detect mode=val model=/content/runs/detect/train/weights/best.pt
data=/content/bottle/data.yaml
```

* for epochs In Google Colab or local PC, training speed depends mostly on the **GPU**, not your Wi-Fi speed.

Case	Typical Speed (for 1000 images)	Comment
A100 GPU (Colab Pro+)	~1–2 min/epoch	Best performance
T4 GPU (free Colab)	~8–12 min/epoch	Normal speed
CPU only	~12–20 min/epoch	Very slow (your case likely)

After executing the above commands
Download the model in the form of best.pt that will be present in bottle folder (location: /content/runs/detect/train/weights/best.pt)

---

5. Model Deployment

We can deploy the model in the Visual Studio Code.
>> Open the Visual Studio Code then download the ultralytics library, opencv library and pytorch library
>> Create a new folder and add the model in the folder (best.pt) file
>> Then add a python file in the folder (deployment.py)
>> Add the below code in the deployment.py file

>> Run the deployment.py file in the Visual Studio Code as

Run Python File

```
from ultralytics import YOLO
import cv2
import torch

# Load models
general_model = YOLO(r'yolov8m.pt')        # general COCO model
bisleri_model = YOLO(r'best9.pt')          # your trained model

device = 'cuda' if torch.cuda.is_available() else 'cpu'
general_model.to(device)
bisleri_model.to(device)
print(f"✅ Using device: {device}")

cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Step 1: Detect general objects
    results_general = general_model(frame, conf=0.5, device=device,
verbose=False)

    for r in results_general:
        boxes = r.boxes
        names = r.names

        for box in boxes:
            cls_id = int(box.cls[0])
            label = names[cls_id].lower()
            x1, y1, x2, y2 = map(int, box.xyxy[0])

            # Check only bottles
            if label == 'bottle':
```

```
                roi = frame[y1:y2, x1:x2]
                if roi.size > 0:
                    # Step 2: Run Bisleri model on cropped bottle
                    results_bisleri = bisleri_model(roi, conf=0.5,
device=device, verbose=False)
                    bisleri_found = False
                    for rb in results_bisleri:
                        for bb in rb.boxes:
                            cls2 = int(bb.cls[0])
                            label2 = rb.names[cls2].lower()
                            if label2 == 'bisleri':
                                bisleri_found = True
    # Draw box
                  color = (0, 255, 0) if bisleri_found else (0, 0, 255)
                   text = "Bisleri" if bisleri_found else "Other Bottle"
                    cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
                    cv2.putText(frame, text, (x1, y1 - 10),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)



# For other objects like person, draw normally
            elif label == 'person':
              cv2.rectangle(frame, (x1, y1), (x2, y2), (255, 255, 0), 2)
                cv2.putText(frame, "Person", (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 0), 2)


   cv2.imshow("Bisleri Detector", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()

cv2.destroyAllWindows()
```

Optimization Techniques

- Use YOLOv8n for faster training on limited GPU
- Freeze lower layers to reduce computation
- Reduce image size during training to accelerate convergence
- Apply early stopping to prevent overfitting
- Use strong augmentation for multi-angle and lighting robustness

---

6. Output

- Trained model weights: best.pt
- Validation metrics (mAP, precision, recall)
- Ability to detect Bisleri bottles from multiple angles, scales, and real-world lighting conditions
- Ready for deployment in automated detection systems

---

7. References

1. Ultralytics YOLOv8 Documentation: https://docs.ultralytics.com
2. Roboflow Dataset and Training Guides: https://app.roboflow.com
3. Python Libraries: ultralytics, roboflow, supervision
4. Custom dataset preparation and augmentation guidelines from project discussion above